

LEARN PYTHON PROGRAMMING – BEGINNER TO MASTER

Section: 7. String and its Methods

Lecture: 74. String Alignment and Padding

Section 1: Lecture Summary

This lecture covers two major categories of string methods in Python: alignment and padding methods, and strip-related methods. It begins with alignment and padding, explaining methods such as `ljust()`, `rjust()`, `center()`, and `zfill()`, which manipulate string width and alignment by adding padding characters. The lecture then moves to strip methods — `lstrip()`, `rstrip()`, and `strip()` — which remove unwanted characters (usually whitespace) from the left, right, or both ends of a string. The concepts are illustrated with examples in a Jupyter notebook environment, emphasizing that string methods return new strings since strings are immutable in Python. Finally, the lecturer encourages extensive practice for mastery and highlights practical use cases like data cleaning.

Section 2: Key Concepts and Explanations

1. Alignment and Padding Methods

These methods adjust the alignment of a string within a given width by padding with spaces or other characters:

- `ljust(width, fillchar=' ')`

- Left-justifies the string within the specified width.
- Pads extra space on the right with the given `fillchar` (defaults to space).
- Returns a new string; original remains unchanged.

- **``rjust(width, fillchar=' ')``**

- Right-justifies the string within the specified width.
- Pads extra space on the left with the given ``fillchar``.
- Returns a new string.

- **``center(width, fillchar=' ')``**

- Centers the string within the specified width.
- Pads extra space equally on both sides with ``fillchar``.
- Returns a new string.

- **``zfill(width)``**

- Pads the string on the left with zeros (``0``) until the string reaches the specified width.
- Particularly useful for padding numbers stored as strings.
- Always pads on the left side.
- Returns a new string.

2. Strip Methods

These methods remove characters from the edges of strings. If no characters are specified, whitespace is removed by default.

- **``lstrip(chars=None)``**

- Removes specified characters from the left (leading) edge of the string.
- If ``chars`` is `None` or omitted, removes leading whitespace.
- Returns a new string.

- ``rstrip(chars=None)``

- Removes specified characters from the right (trailing) edge.
- Defaults to removing trailing whitespace if ``chars`` is not specified.
- Returns a new string.

- ``strip(chars=None)``

- Removes specified characters from both the left and right edges.
- Defaults to removing both leading and trailing whitespace.
- Returns a new string.

- When a string of characters is passed to these methods (e.g., ``strip('#! $')``), it removes **any combination** of these characters from the respective ends **until it encounters a character not in the set**. It does not remove characters from the middle of the string.

3. Immutability of Strings

- All these methods do not modify the original string since strings in Python are immutable.
- They return a new string reflecting the operation.

4. Practical Usage Tips

- Alignment and padding are useful in formatting outputs like tables, logs, or reports.
- Strip methods are essential in data cleaning — removing unwanted spaces or junk characters from input data.

Section 3: Example Code and Use Cases

Create a base string

Left justify with width=7, padding with '*'

Right justify with width=7, default padding (space)

Right justify with width=7, padding with '-'

Center with width=7, default padding (space)

Center with width=7, padding with '*'

Zero fill with width=7

String with leading spaces

Remove leading spaces using lstrip()

String with leading special characters

Remove leading '\$' signs using lstrip()

String with trailing special characters

Remove trailing '!' characters using rstrip()

String with characters on both sides

Remove " from both ends using strip()

String with multiple junk characters on both sides

Remove multiple characters from both ends

```
s = 'Hello'

x = s.ljust(7, '*')
print(x) # Output: Hello**

x = s.rjust(7)
print(f"{{x}}") # Output: ' Hello' (spaces on left)

x = s.rjust(7, '-')
print(x) # Output: --Hello

x = s.center(7)
print(f"{{x}}") # Output: ' Hello ' (space on both sides)

x = s.center(7, '*')
```



```
print(x) # Output: *Hello*

x = s.zfill(7)
print(x) # Output: 00Hello

s = ' Hello'

x = s.lstrip()
print(f'"{x}"') # Output: 'Hello'

s = '$$Hello'

x = s.lstrip('$')
print(x) # Output: Hello

s = 'Hello!!'

x = s.rstrip('!!')
print(x) # Output: Hello

s = '#Hello#'

x = s.strip('#')
print(x) # Output: Hello

s = '#!Hello $ x'

x = s.strip('#! $x')
print(x) # Output: Hello
```

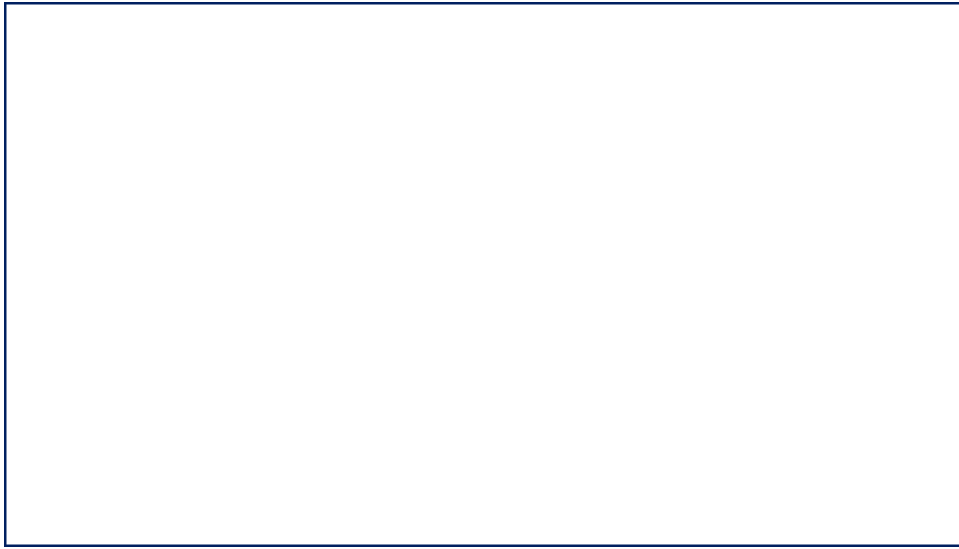
Explanation: Each code snippet demonstrates a string method for alignment, padding, or stripping, showcasing how to control string formatting or clean unwanted characters from strings. Use of quotes in prints helps to visualize spaces.

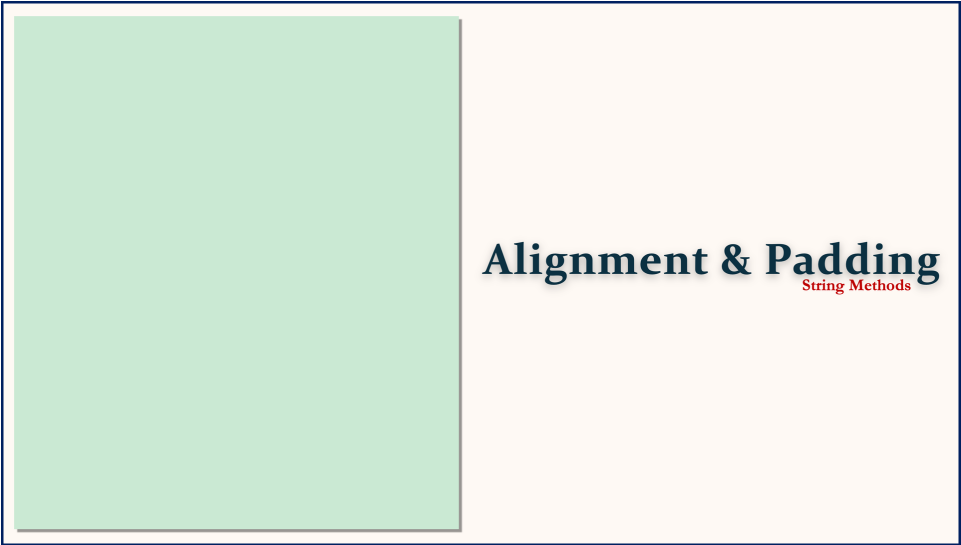
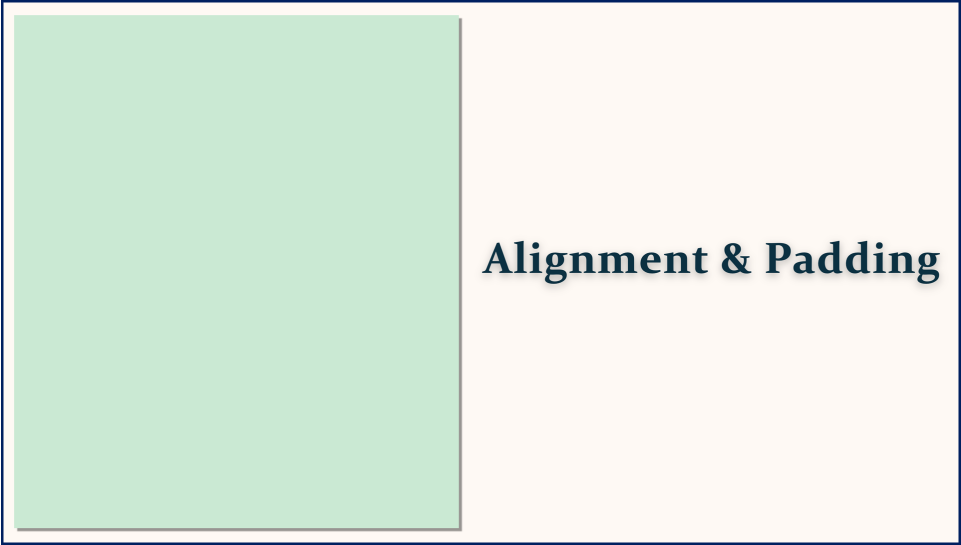
Section 4: Key Takeaways

- String alignment and padding methods (`ljust()`, `rjust()`, `center()`, `zfill()`) adjust string width and alignment by adding padding characters around the string.
- Strip methods (`lstrip()`, `rstrip()`, `strip()`) remove unwanted characters from the left side, right side, or both ends of a string, typically whitespace or specified characters.
- All these methods return a **new string**; the original string remains unchanged due to string immutability in Python.
- You can specify a single or multiple characters in strip methods to remove sequences of unwanted leading/trailing characters.
- These methods are crucial for formatting output and cleaning data, and practicing them extensively improves your coding efficiency and data handling.

Lecture in Slides

Please Note: This document presents a curated selection of slides from the lecture; others have been excluded for conciseness.





Alignment & Padding
String Methods

Alignment & Padding
String Methods

- **Alignment & Padding**

Alignment & Padding

String Methods

- Alignment & Padding
- Strip Methods

Alignment & Padding

String Methods

ljust(width , fillChar)

Alignment & Padding

String Methods

ljust(width , fillChar)

rjust(width , fillChar)

s

H	e	l	l	o						
---	---	---	---	---	--	--	--	--	--	--

Alignment & Padding

String Methods

ljust(width , fillChar)

rjust(width , fillChar)

s

						H	e	l	l	o
--	--	--	--	--	--	---	---	---	---	---

Alignment & Padding

String Methods

ljust(width , fillChar)

rjust(width , fillChar)

center(width , fillChar)

s

H	e	l	l	o					
---	---	---	---	---	--	--	--	--	--

Alignment & Padding

String Methods

ljust(width , fillChar)

rjust(width , fillChar)

center(width , fillChar)

s

			H	e	l	l	o			
--	--	--	---	---	---	---	---	--	--	--

Alignment & Padding

String Methods

ljust(width , fillChar)

rjust(width , fillChar)

center(width , fillChar)

zfill(width)

s

H	e	l	l	o
---	---	---	---	---

Alignment & Padding

String Methods

ljust(width , fillChar)

rjust(width , fillChar)

center(width , fillChar)

zfill(width)

s 0 0 0 0 0 0 H e l l o

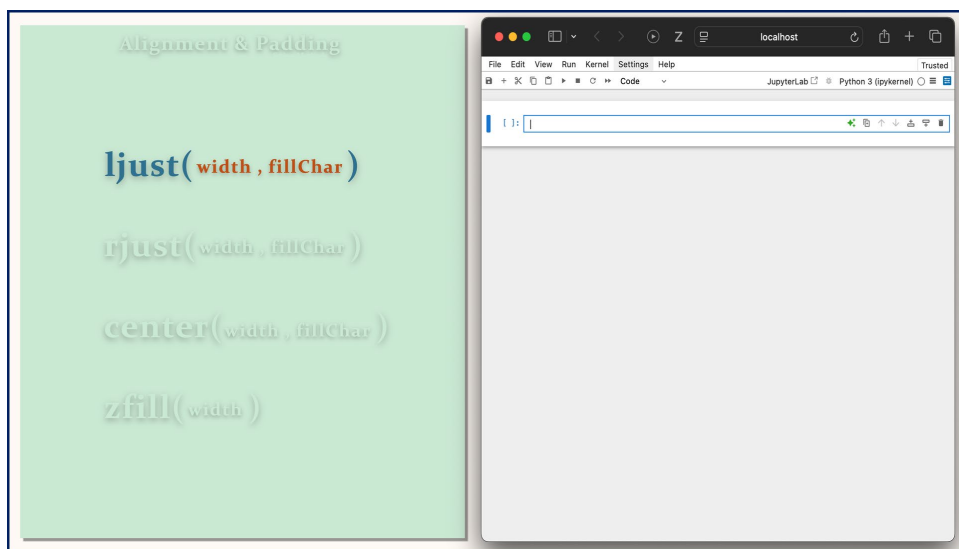
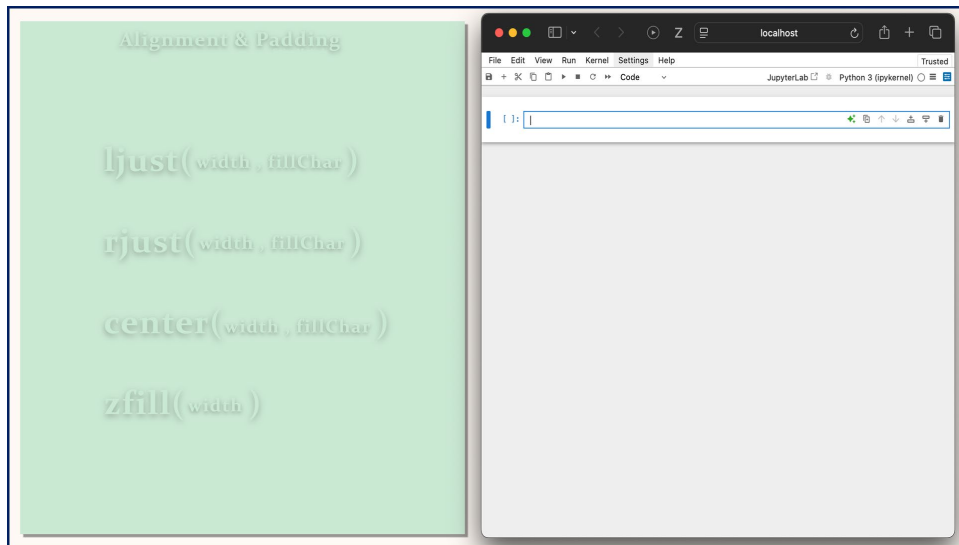
Alignment & Padding

ljust(width , fillChar)

rjust(width , fillChar)

center(width , fillChar)

zfill(width)



Alignment & Padding

ljust(width , fillChar)

rjust(width , fillChar)

center(width , fillChar)

zfill(width)

The JupyterLab interface shows a code cell with the line `s = "Hello"`. The output cell displays a string array visualization. It consists of a large 'S' on the left, followed by a horizontal row of five boxes. Above each box is an index number from 0 to 4. The boxes contain the characters 'H', 'e', 'l', 'l', and 'o' in red text.

0	1	2	3	4
H	e	l	l	o

Alignment & Padding

ljust(width , fillChar)

rjust(width , fillChar)

center(width , fillChar)

zfill(width)

The JupyterLab interface shows a code cell with the line `s = "Hello"`. The output cell displays a single character array visualization. It consists of a large 'S' on the left, followed by a single box containing the character 'H' in red text.

H

Alignment & Padding

ljust(width , fillChar)

rjust(width , fillChar)

center(width , fillChar)

zfill(width)

```
s = "Hello"
```

	0	1	2	3	4
s	H	e	l	l	o

s.ljust()

Alignment & Padding

ljust(width , fillChar)

rjust(width , fillChar)

center(width , fillChar)

zfill(width)

```
s = "Hello"
```

	0	1	2	3	4
s	H	e	l	l	o

s.ljust(7)

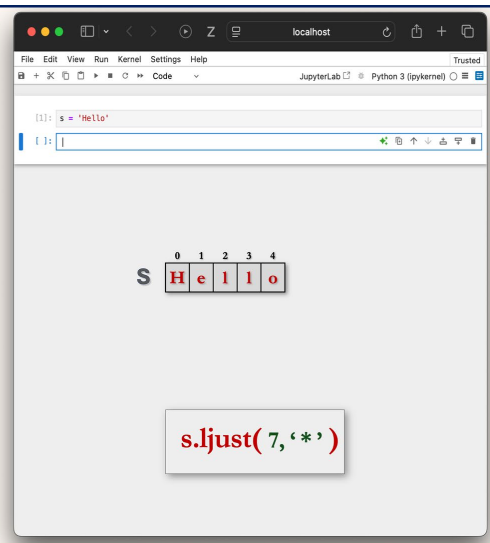
Alignment & Padding

ljust(width, fillChar)

rjust(width, fillChar)

center(width, fillChar)

zfill(width)



```
[1]: s = 'Hello'
```

```
[ ]:
```

S 0 1 2 3 4
 H e l l o

s.ljust(7, '*')

Alignment & Padding

ljust(width , fillChar)

rjust(width , fillChar)

center(width , fillChar)

zfill(width)

```
s = 'Hello'
```

	0	1	2	3	4
S	H	e	l	l	o

```
x = s.ljust(7, '*')
```

Alignment & Padding

ljust(width , fillChar)

rjust(width , fillChar)

center(width , fillChar)

zfill(width)

```
s = 'Hello'
```

```
x = s.ljust(7, '*')
```

	0	1	2	3	4
S	H	e	l	l	o

	0	1	2	3	4	5	6
x	H	e	l	l	o	*	*

```
x = s.ljust(7, '*')
```


Alignment & Padding

ljust(width , fillChar)

rjust(width , fillChar)

center(width , fillChar)

zfill(width)

```
[1]: s = 'Hello'
[3]: x = s.ljust(7,'*')
[5]: print(x)
Hello**
```

1:

	0	1	2	3	4
S	H	e	l	l	o

	0	1	2	3	4	5	6
x	H	e	l	l	o	*	*

x = s.ljust(7,'*')

Alignment & Padding

ljust(width , fillChar)

rjust(width , fillChar)

center(width , fillChar)

zfill(width)

```
[1]: s = 'Hello'
1:
```

	0	1	2	3	4
S	H	e	l	l	o

Alignment & Padding

`ljust(width, fillChar)`

`rjust(width, fillChar)`

`center(width, fillChar)`

`zfill(width)`

localhost

File Edit View Run Kernel Settings Help

JupyterLab Python 3 (ipykernel)

```
[1]: s = 'Hello'
```

```
[1]:
```

S

0	1	2	3	4
H	e	l	l	o

Alignment & Padding

`ljust(width, fillChar)`

`rjust(width, fillChar)`

`center(width, fillChar)`

`zfill(width)`

localhost

File Edit View Run Kernel Settings Help

JupyterLab Python 3 (ipykernel)

```
[1]: s = 'Hello'
```

```
[1]:
```

S

0	1	2	3	4
H	e	l	l	o

```
x = s.rjust(7)
```

Alignment & Padding

`ljust(width, fillChar)`

`rjust(width, fillChar)`

`center(width, fillChar)`

`zfill(width)`

```
s = "Hello"
x = s.rjust(7)
```

S	0	1	2	3	4
	H	e	l	l	o

x	0	1	2	3	4	5	6
			H	e	l	l	o

`x = s.rjust(7)`

Alignment & Padding

`ljust(width, fillChar)`

`rjust(width, fillChar)`

`center(width, fillChar)`

`zfill(width)`

```
s = "Hello"
x = s.rjust(7)
print(x)
```

```
Hello
```

S	0	1	2	3	4
	H	e	l	l	o

x	0	1	2	3	4	5	6
			H	e	l	l	o

`x = s.rjust(7)`

Alignment & Padding

`ljust(width, fillChar)`

`rjust(width, fillChar)`

`center(width, fillChar)`

`zfill(width)`

```
s = "Hello"
```

	0	1	2	3	4
s	H	e	l	l	o

`x = s.rjust(7, '-')`

Alignment & Padding

`ljust(width, fillChar)`

`rjust(width, fillChar)`

`center(width, fillChar)`

`zfill(width)`

```
s = "Hello"
```

```
x = s.rjust(7, '-')
```

	0	1	2	3	4
s	H	e	l	l	o

	0	1	2	3	4	5	6
x	-	-	H	e	l	l	o

`x = s.rjust(7, '-')`

Alignment & Padding

`ljust(width, fillChar)`

`rjust(width, fillChar)`

`center(width, fillChar)`

`zfill(width)`

```
File Edit View Run Kernel Settings Help Trusted
JupyterLab Python 3 (pykernel)

[1]: s = 'Hello'
[3]: x = s.rjust(7, '-')
[5]: print(x)
--Hello

[ ]:
```

S

0	1	2	3	4
H	e	l	l	o

x

0	1	2	3	4	5	6
-	-	H	e	l	l	o

`x = s.rjust(7, '-')`

Alignment & Padding

`ljust(width, fillChar)`

`rjust(width, fillChar)`

`center(width, fillChar)`

`zfill(width)`

The JupyterLab interface shows a code cell with the line `s = 'Hello'`. The output cell displays a string array visualization. It consists of a label 'S' followed by a horizontal array of five boxes. Above the boxes are indices 0, 1, 2, 3, and 4. The boxes contain the characters 'H', 'e', 'l', 'l', and 'o' respectively. The 'e' and 'l' boxes are highlighted in red.

Alignment & Padding

`ljust(width, fillChar)`

`rjust(width, fillChar)`

`center(width, fillChar)`

`zfill(width)`

The JupyterLab interface shows a code cell with the line `s = 'Hello'`. The output cell displays a string array visualization, identical to the one in the first image. Below the visualization, there is a code snippet box containing the line `x = s.center(7)`.

Alignment & Padding

`ljust(width, fillChar)`

`rjust(width, fillChar)`

`center(width, fillChar)`

`zfill(width)`

```
s = 'Hello'
x = s.center(7)
```

`x`

	0	1	2	3	4
<code>s</code>	H	e	l	l	o

	0	1	2	3	4	5	6
<code>x</code>		H	e	l	l	o	

`x = s.center(7)`

Alignment & Padding

`ljust(width, fillChar)`

`rjust(width, fillChar)`

`center(width, fillChar)`

`zfill(width)`

```
s = 'Hello'
x = s.center(7)
print(x)
```

`x`

	0	1	2	3	4
<code>s</code>	H	e	l	l	o

	0	1	2	3	4	5	6
<code>x</code>		H	e	l	l	o	

`x = s.center(7)`

Alignment & Padding

`ljust(width, fillChar)`

`rjust(width, fillChar)`

`center(width, fillChar)`

`zfill(width)`

```
s = 'Hello'
```

	0	1	2	3	4
S	H	e	l	l	o

`x = s.center(7, '*')`

Alignment & Padding

`ljust(width, fillChar)`

`rjust(width, fillChar)`

`center(width, fillChar)`

`zfill(width)`

```
s = 'Hello'
```

	0	1	2	3	4
S	H	e	l	l	o

```
x = s.center(7, '*')
```

	0	1	2	3	4	5	6
X	*	H	e	l	l	o	*

`x = s.center(7, '*')`

Alignment & Padding

`ljust(width, fillChar)`

`rjust(width, fillChar)`

`center(width, fillChar)`

`zfill(width)`

localhost

File Edit View Run Kernel Settings Help

Trusted

JupyterLab Python 3 (ipykernel)

```
[1]: s = "Hello"
[3]: x = s.center(7, '*')
[5]: print(x)
*Hello*
```

[1]:

S

0

1

2

3

4

H

e

l

l

o

X

0

1

2

3

4

5

6

*

H

e

l

l

o

*

x = s.center(7, '*')

Alignment & Padding

`ljust(width, fillChar)`

`rjust(width, fillChar)`

`center(width, fillChar)`

`zfill(width)`

localhost

File Edit View Run Kernel Settings Help

Trusted

JupyterLab Python 3 (ipykernel)

```
[1]: s = "Hello"
```

[1]:

S

0

1

2

3

4

H

e

l

l

o

Alignment & Padding

Sung Min Choi

ljust(width, fillChar)

rjust(width, fillChar)

center(width, fillChar)

zfill(width)

localhost

File Edit View Run Kernel Settings Help

Trusted

JupyterLab Python 3 (ipykernel)

[1]: s = "Hello"

[]: |

S

0

1

2

3

4

H

e

l

l

o

x = s.zfill(7)

Alignment & Padding

Sung Min Choi

ljust(width, fillChar)

rjust(width, fillChar)

center(width, fillChar)

zfill(width)

localhost

File Edit View Run Kernel Settings Help

Trusted

JupyterLab Python 3 (ipykernel)

[1]: s = "Hello"

[2]: x = s.zfill(7)

[]: |

S

0

1

2

3

4

H

e

l

l

o

x

0

1

2

3

4

5

6

0

0

H

e

l

l

o

x = s.zfill(7)

Alignment & Padding

`ljust(width, fillChar)`

`rjust(width, fillChar)`

`center(width, fillChar)`

`zfill(width)`

```
File Edit View Run Kernel Settings Help Trusted
JupyterLab Python 3 (ipykernel)
```

```
[1]: s = 'Hello'
[3]: x = s.zfill(7)
[5]: print(x)
00Hello
```

S

0	1	2	3	4
H	e	l	l	o

x

0	1	2	3	4	5	6
0	0	H	e	l	l	o

`x = s.zfill(7)`

Strip Methods

String Methods

Strip Methods

String Methods

String Methods

show

String Methods

1. ...

He

Strip Methods

String Methods

lstrip(chars)

s H e l l o

Strip Methods

String Methods

lstrip(chars)

rstrip(chars)

s H e l l o - - - - -

Strip Methods

String Methods

lstrip(chars)

rstrip(chars)

s

H	e	l	l	o
---	---	---	---	---

Strip Methods

String Methods

lstrip(chars)

rstrip(chars)

strip(chars)

s

			H	e	l	l	o		
--	--	--	---	---	---	---	---	--	--

Strip Methods

lstrip(chars)

rstrip(chars)

strip(chars)

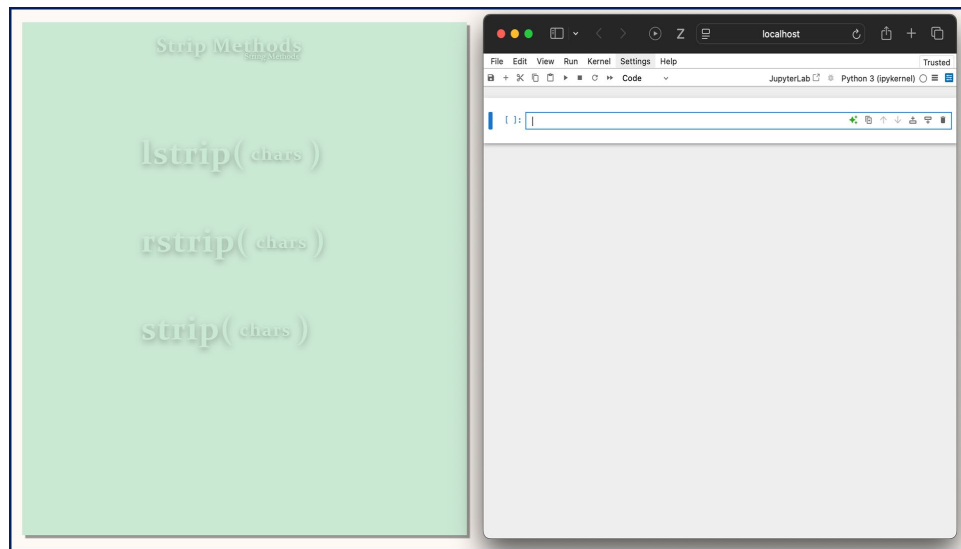
sHello

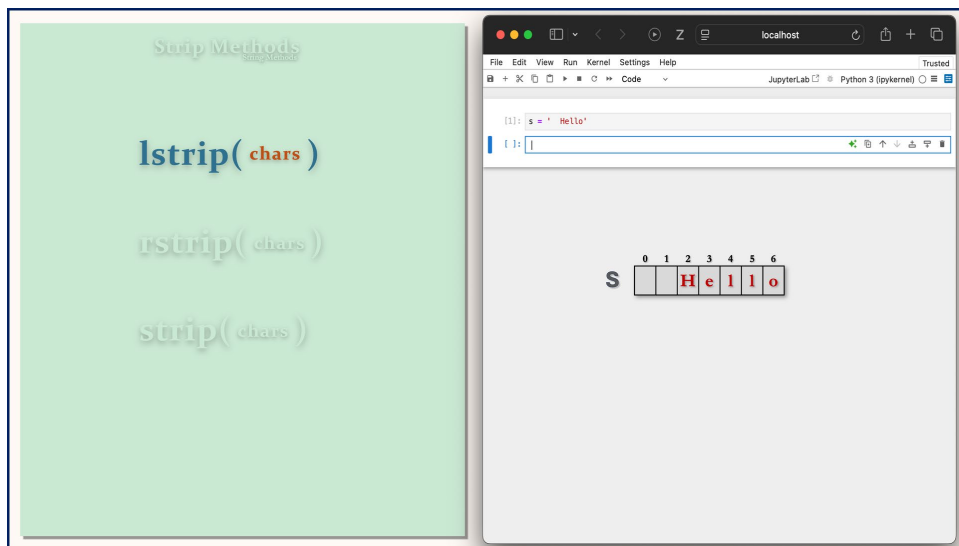
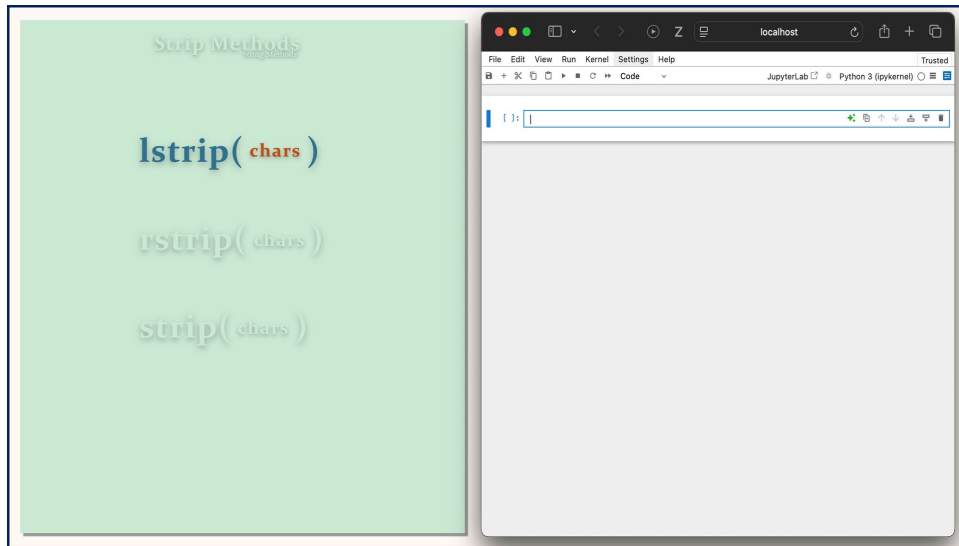
Strip Methods

lstrip(chars)

rstrip(chars)

strip(chars)





Strip Methods

`lstrip(chars)`

`rstrip(chars)`

`strip(chars)`

localhost

File Edit View Run Kernel Settings Help

Trusted

JupyterLab Python 3 (ipykernel)

```
[1]: s = ' Hello'
```

```
[ ]:
```

	0	1	2	3	4	5	6
S			H	e	l	l	o

Strip Methods

`lstrip(chars)`

`rstrip(chars)`

`strip(chars)`

localhost

File Edit View Run Kernel Settings Help

Trusted

JupyterLab Python 3 (ipykernel)

```
[1]: s = ' Hello'
```

```
[ ]:
```

	0	1	2	3	4	5	6
S			H	e	l	l	o

`x = s.lstrip( )`

Strip Methods

`lstrip(chars)`

`rstrip(chars)`

`strip(chars)`

```
[1]: s = ' Hello'
[3]: x = s.lstrip()

In [ ]:
```

S

0123456

H

e

l

l

o

x

01234

H

e

l

l

o

`x = s.lstrip( )`

Strip Methods

`lstrip(chars)`

`rstrip(chars)`

`strip(chars)`

```
[1]: s = ' Hello'
[3]: x = s.lstrip()
[5]: print(x)
Hello

In [ ]:
```

S

0123456

H

e

l

l

o

x

01234

H

e

l

l

o

`x = s.lstrip( )`

Strip Methods

`lstrip(chars)`

`rstrip(chars)`

`strip(chars)`

```
[1]: s = "$$Hello"
[5]: x = s.lstrip('$')
[ ]:
```

S

0123456

\$ \$ H e l l o

x

01234

H e l l o

`x = s.lstrip('$')`

localhost

File Edit View Run Kernel Settings Help

Trusted

JupyterLab Python 3 (ipykernel)

```
[1]: s = "$$Hello"
[5]: x = s.lstrip('$')
[ ]:
```

S

0123456

\$ \$ H e l l o

x

01234

H e l l o

`x = s.lstrip('$')`

Strip Methods

`lstrip(chars)`

`rstrip(chars)`

`strip(chars)`

```
[1]: s = "$$Hello"
[5]: x = s.lstrip('$')
[7]: print(x)
Hello
[ ]:
```

S

0123456

\$ \$ H e l l o

x

01234

H e l l o

`x = s.lstrip('$')`

localhost

File Edit View Run Kernel Settings Help

Trusted

JupyterLab Python 3 (ipykernel)

```
[1]: s = "$$Hello"
[5]: x = s.lstrip('$')
[7]: print(x)
Hello
[ ]:
```

S

0123456

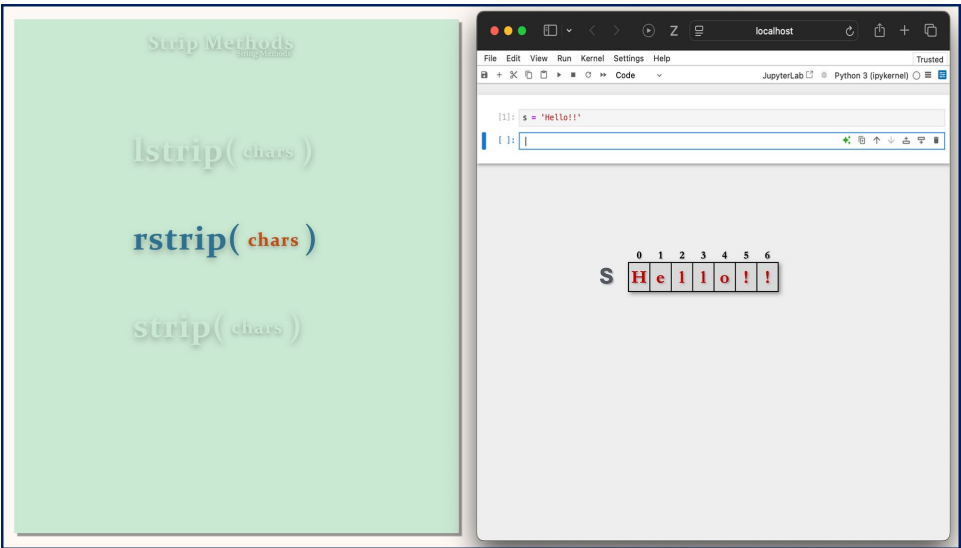
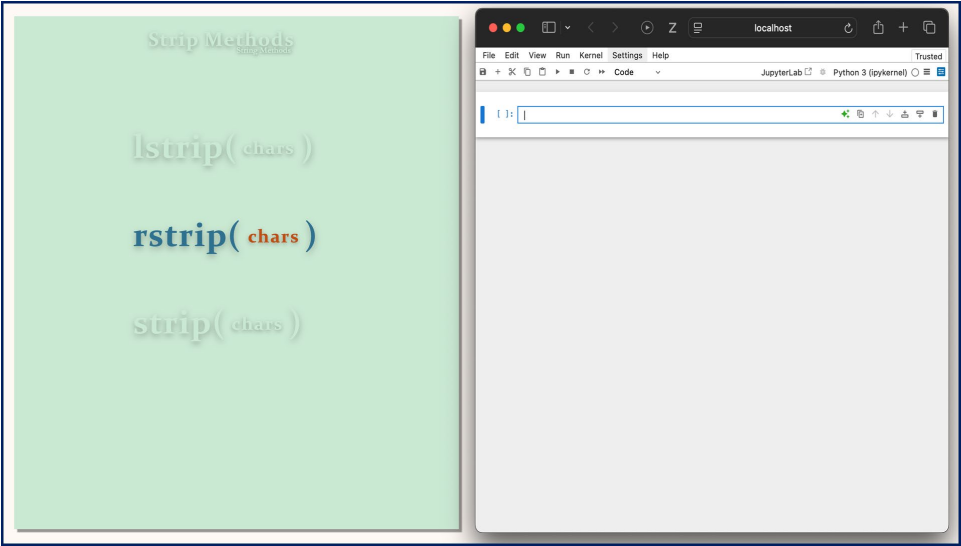
\$ \$ H e l l o

x

01234

H e l l o

`x = s.lstrip('$')`



Strip Methods

`rstrip( chars )`

`rstrip( chars )`

`strip( chars )`

localhost

File Edit View Run Kernel Settings Help

JupyterLab Python 3 (ipykernel)

```
[1]: s = "Hello!"
[3]: print(s)
Hello!
```

S

0	1	2	3	4	5	6
H	e	l	l	o	!	!

Strip Methods

`rstrip( chars )`

`strip( chars )`

```
[1]: s = "Hello!!"
[3]: print(s)
Hello!!
[5]: x = s.rstrip('!')
[ ]: |
```

	0	1	2	3	4	5	6
S	H	e	l	l	o	!	!

	0	1	2	3	4
x	H	e	l	l	o

`x = s.rstrip('!')`

Strip Methods

`rstrip( chars )`

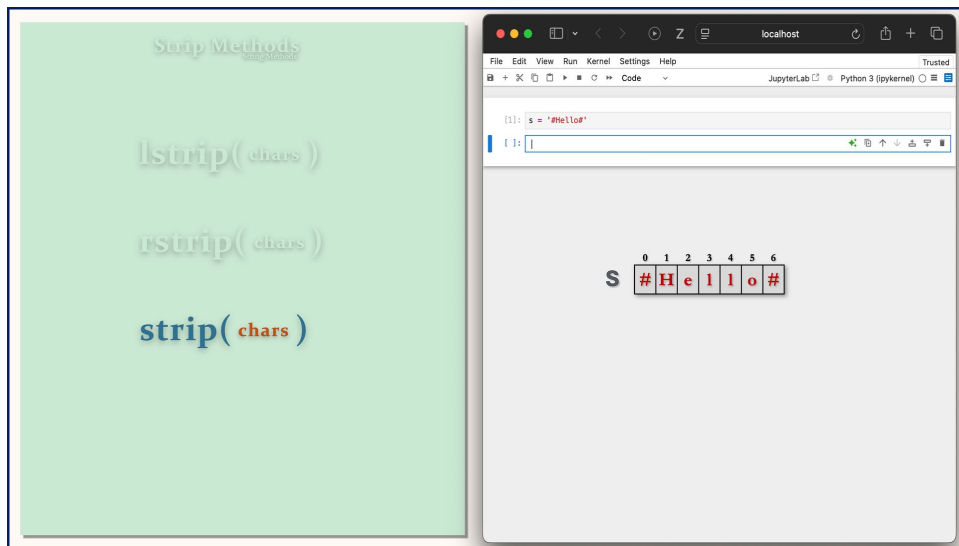
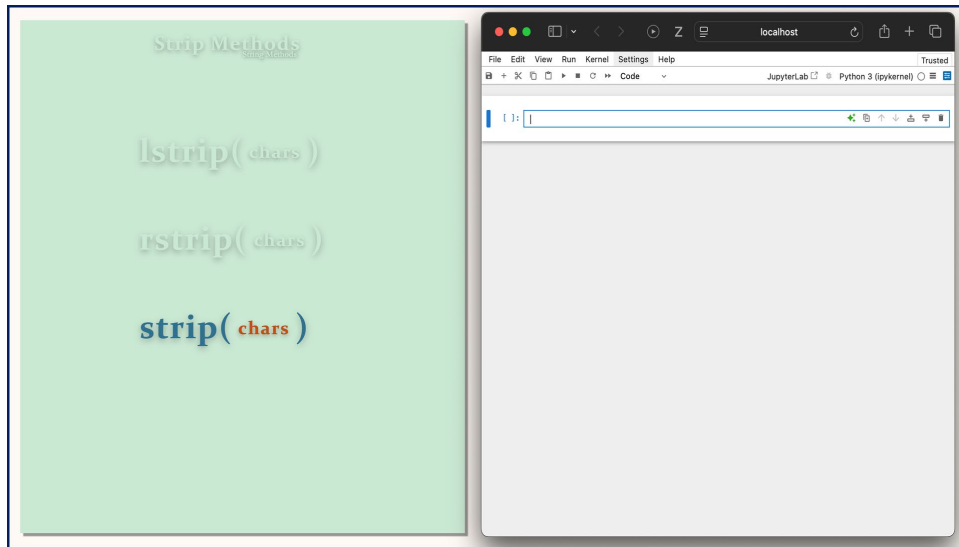
`strip( chars )`

```
[1]: s = "Hello!!"
[3]: print(s)
Hello!!
[5]: x = s.rstrip('!')
[7]: print(x)
Hello
[ ]: |
```

	0	1	2	3	4	5	6
S	H	e	l	l	o	!	!

	0	1	2	3	4
x	H	e	l	l	o

`x = s.rstrip('!')`



Strip Methods

strip(chars)

rstrip(chars)

strip(chars)

localhost

File Edit View Run Kernel Settings Help

Trusted

JupyterLab Python 3 (ipykernel)

```
[1]: s = "#Hello#"
[3]: print(s)
#Hello#
```

S

0	1	2	3	4	5	6
#	H	e	l	l	o	#

Strip Methods

strip(chars)

rstrip(chars)

strip(chars)

localhost

File Edit View Run Kernel Settings Help

Trusted

JupyterLab Python 3 (ipykernel)

```
[1]: s = "#Hello#"
[3]: print(s)
#Hello#
```

S

0	1	2	3	4	5	6
#	H	e	l	l	o	#

Strip Methods

strip(chars)

rstrip(chars)

strip(chars)

localhost

File Edit View Run Kernel Settings Help

Trusted

JupyterLab Python 3 (ipykernel)

[1]: s = "Hello#"

[3]: print(s)

#Hello#

[]:

S # H e l l o #

x = s.strip("#")

Strip Methods

strip(chars)

rstrip(chars)

strip(chars)

localhost

File Edit View Run Kernel Settings Help

Trusted

JupyterLab Python 3 (ipykernel)

[1]: s = "Hello#"

[3]: print(s)

#Hello#

[5]: x = s.strip("#")

[]:

S # H e l l o #

x H e l l o

x = s.strip("#")

Strip Methods

strip(chars)

rstrip(chars)

strip(chars)

```
[1]: s = "#Hello#"
[3]: print(s)
#Hello#
[5]: x = s.strip("#")
[7]: print(x)
Hello
```

S

0	1	2	3	4	5	6
#	H	e	l	l	o	#

x

0	1	2	3	4
H	e	l	l	o

x = s.strip("#")

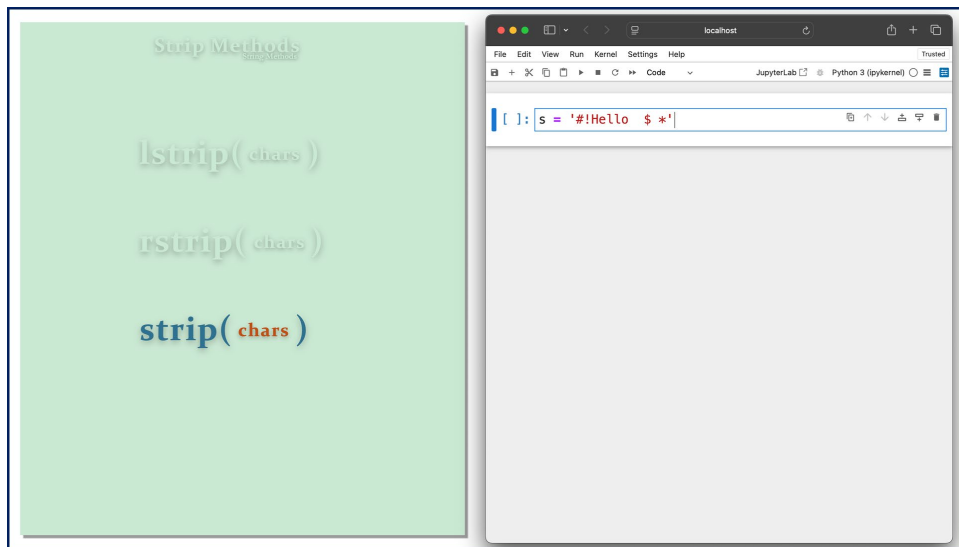
Strip Methods

strip(chars)

rstrip(chars)

strip(chars)

```
[ ]:
```



Strip Methods

`rstrip(chars)`

`rstrip(chars)`

`strip(chars)`

```
[1]: s = '#!Hello $ *'
```

```
[ ]:
```

	0	1	2	3	4	5	6	7	8	9	10	11
S	#	!	H	e	l	l	o			\$		*

Strip Methods

`rstrip(chars)`

`rstrip(chars)`

`strip(chars)`

```
[1]: s = '#!Hello $ *'
```

```
[ ]:
```

	0	1	2	3	4	5	6	7	8	9	10	11
S	#	!	H	e	l	l	o			\$		*

Strip Methods

Strip Method

`lstrip(chars)`

`rstrip(chars)`

`strip(chars)`

```
[1]: s = '#!Hello $ *'
```

```
[ ]:
```

	0	1	2	3	4	5	6	7	8	9	10	11
S	#	!	H	e	l	l	o			\$		*

`x = s.strip( )`

Strip Methods

Strip Method

`lstrip(chars)`

`rstrip(chars)`

`strip(chars)`

```
[1]: s = '#!Hello $ *'
```

```
[ ]:
```

	0	1	2	3	4	5	6	7	8	9	10	11
S	#	!	H	e	l	l	o			\$		*

`x = s.strip( '#' )`

Strip Methods

strip(chars)

rstrip(chars)

strip(chars)

```
[1]: s = '#!Hello $ *'
```

```
[ ]:
```

0	1	2	3	4	5	6	7	8	9	10	11
S	#	!	H	e	l	l	o			\$	*

x = s.strip('#!')

Strip Methods

strip(chars)

rstrip(chars)

strip(chars)

```
[1]: s = '#!Hello $ *'
```

```
[ ]:
```

0	1	2	3	4	5	6	7	8	9	10	11
S	#	!	H	e	l	l	o			\$	*

x = s.strip('#!')

Strip Methods

`rstrip(chars)`

`rstrip(chars)`

`strip(chars)`

```
[1]: s = '#!Hello $*'
```

```
[ ]:
```

S		0	1	2	3	4	5	6	7	8	9	10	11
	#	!	H	e	l	l	o					\$	*

`x = s.strip('#! $*')`

Strip Methods

`rstrip(chars)`

`rstrip(chars)`

`strip(chars)`

```
[1]: s = '#!Hello $*'
```

```
[ ]:
```

S		0	1	2	3	4	5	6	7	8	9	10	11
	#	!	H	e	l	l	o					\$	*

`x = s.strip('#! $*')`

Strip Methods

strip(chars)

rstrip(chars)

strip(chars)

```
[1]: s = '#!Hello $ *'
[2]: x = s.strip('! $*')
[ ]:
```

S	0	1	2	3	4	5	6	7	8	9	10	11
	#	!	H	e	l	l	o			\$		*

X	H	e	l	l	o
---	---	---	---	---	---

x = s.strip('! \$*')

Strip Methods

strip(chars)

rstrip(chars)

strip(chars)

```
[1]: s = '#!Hello $ *'
[2]: x = s.strip('! $*')
[3]: print(x)
Hello
[ ]:
```

S	0	1	2	3	4	5	6	7	8	9	10	11
	#	!	H	e	l	l	o			\$		*

X	H	e	l	l	o
---	---	---	---	---	---

x = s.strip('! \$*')



Strip Methods

`rstrip( chars )`

`rstrip( chars )`

`strip( chars )`

```
[1]: s = '#!Hello $ *'
[2]: print(s)
#!Hello $ *
```

	0	1	2	3	4	5	6	7	8	9	10	11
S	#	!	H	e	l	l	o			\$		*

```
x = s.strip('$')
```

Strip Methods

`rstrip( chars )`

`rstrip( chars )`

`strip( chars )`

```
[1]: s = '#!Hello $ *'
[2]: print(s)
#!Hello $ *
```

	0	1	2	3	4	5	6	7	8	9	10	11
S	#	!	H	e	l	l	o			\$		*

```
x = s.strip('$#!')
```

Strip Methods

Strip Method

`rstrip(chars)`

`rstrip(chars)`

`strip(chars)`

localhost

Untitled1

File Edit View Run Kernel Settings Help Trusted

+ - X Z

Code

JupyterLab Python 3 (ipykernel)

[1]: `s = '#!Hello $ *'`

[2]: `print(s)`

`#!Hello $ *`

[]:

0 1 2 3 4 5 6 7 8 9 10 11

S `# ! H e l l o $ *`

`x = s.strip('$#! ')`

Strip Methods

Strip Method

`rstrip(chars)`

`rstrip(chars)`

`strip(chars)`

localhost

Untitled1

File Edit View Run Kernel Settings Help Trusted

+ - X Z

Code

JupyterLab Python 3 (ipykernel)

[1]: `s = '#!Hello $ *'`

[2]: `print(s)`

`#!Hello $ *`

[]:

0 1 2 3 4 5 6 7 8 9 10 11

S `# ! H e l l o $ *`

`x = s.strip('$#! *')`

Strip Methods
String Methods

rstrip(*chars*)

rstrip(*chars*)

strip(*chars*)

localhost

Untitled1

File Edit View Run Kernel Settings Help Trusted

Python 3 (ipykernel)

```
[1]: s = '#!Hello $ *'
[2]: print(s)
#!Hello $ *
[3]: x = s.strip(' $#! *')
```

S

0 1 2 3 4 5 6 7 8 9 10 11

! H e l l o \$ *

X

0 1 2 3 4

H e l l o

x = s.strip(' \$#! *')

Strip Methods
String Methods

rstrip(*chars*)

rstrip(*chars*)

strip(*chars*)

localhost

Untitled1

File Edit View Run Kernel Settings Help Trusted

Python 3 (ipykernel)

```
[1]: s = '#!Hello $ *'
[2]: print(s)
#!Hello $ *
[3]: x = s.strip(' $#! *')
[4]: print(x)
Hello
```

S

0 1 2 3 4 5 6 7 8 9 10 11

! H e l l o \$ *

X

0 1 2 3 4

H e l l o

x = s.strip(' \$#! *')

